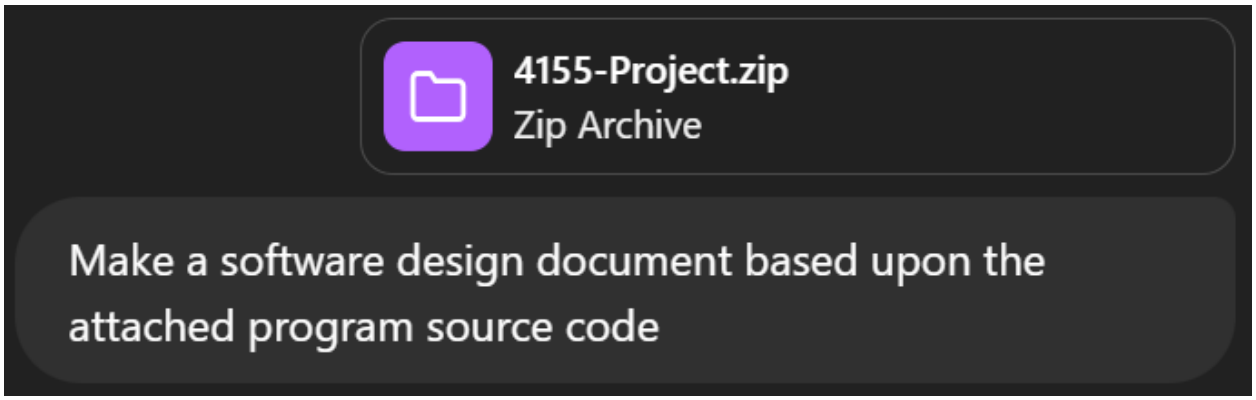
Software Design Document (SDD)

Generated by ChatGPT

Prompt:



Project Title: Gatherly

---

1. Overview

Project Summary
Gatherly is a web-based platform designed to streamline the process of discovering, booking, and managing event venues. It allows users to browse through various listings, view venue details, book reservations, communicate with venue owners, leave reviews, and manage notifications and preferences.

Key Features

- User registration and login
- Venue search, creation, and editing
- Booking management system
- Review and rating functionality
- Messaging between users

- Notifications for activity updates
- Shopping cart for bookings
- Favorites system

---

2. System Architecture

Architecture Overview

```
Frontend (Vue.js)  <-->  Backend (Node.js + Express)  <-->  Database (MongoDB)
```

Technologies Used

- **Frontend**: Vue.js, JavaScript, CSS
- **Backend**: Node.js, Express.js
- **Database**: MongoDB with Mongoose ODM
- **Testing**: Vitest (frontend), Jest (backend)
- **Build Tools**: Vite

---

3. Frontend Design

Structure Overview

- `App.vue`: Root Vue component
- `main.js`: App initialization and router integration
- `router.js`: Defines client-side routing

Main Components

- **Authentication & User**:
    - `LoginModal.vue`, `SignupModal.vue`, `SettingsPage.vue`
- **Venue Management**:
    - `VenueCard.vue`, `VenueDetailsPage.vue`, `CreateVenue.vue`, `EditVenue.vue`
- **Booking & Cart**:
    - `Booking.vue`, `ShoppingCart.vue`
- **Messaging & Notifications**:
    - `Messages.vue`, `NotifPage.vue`
- **Navigation & Layout**:
    - `NavBar.vue`, `FooterComponent.vue`, `LandingPage.vue`, `ErrorPage.vue`

State Management

- `cartStore.js`: Manages booking cart state
- `userStore.js`: Handles user session and profile state
- `venueStore.js`: Stores venue-related data

---

4. Backend Design

Core Structure

- **Routes**: API entrypoints (e.g., `venueRoutes.js`, `userRoutes.js`)
- **Models**: Mongoose schemas for domain entities
- **Middleware**: Authentication, input validation, and logic encapsulation

- **Testing**: Unit and integration tests

Main Models (Schemas)

- `User`: Name, email, password, favorites
- `Venue`: Name, description, images, location, price
- `Booking`: Venue, user, date, status
- `Review`: User, venue, rating, comment
- `Message`: Sender, recipient, content
- `Notification`: Type, recipient, message

Middleware Example

- `venueMiddleware.js`: Handles venue-specific authorization and validation

---

5. API Overview (Selected Routes)

Method	Endpoint	Description
GET	`/venues/`	Retrieve list of all venues
POST	`/venues/`	Create a new venue
GET	`/venues/:id`	View specific venue details

POST	`/users/login`	Authenticate user credentials
POST	`/bookings/`	Create a new booking
GET	`/bookings/:userId`	List bookings for a user
POST	`/favorites/`	Add venue to user's favorites
GET	`/messages/`	Retrieve user messages

---

6. Database Design

MongoDB Collections

- `users`: Registered user data
- `venues`: Venue listings and details
- `bookings`: Booking records
- `reviews`: Ratings and comments
- `messages`: User communication
- `notifications`: Activity alerts

Relations

- Bookings and reviews reference both users and venues
- Favorites are embedded within the `users` collection
- Messages are linked via sender and recipient IDs

---

7. Testing Strategy

Frontend Testing (Vitest)

- Component testing:
    - `CreateVenue.test.js`
    - `Navbar.test.js`

Backend Testing (Jest)

- Middleware logic and route handlers:
    - `userMiddleware.test.js`
    - `venuecontroller.test.js`

Goals

- Ensure core functionalities are intact
- Validate business logic and error handling
- Mock database interactions where needed

---

8. Security Considerations

- Password hashing (e.g., bcrypt assumed)
- Input sanitization and validation middleware
- Role-based authorization for sensitive endpoints
- Secure session and token management (JWT presumed)
- API rate limiting and error handling best practices

---

9. Future Improvements

- Role-based access controls for admin functions
- Real-time chat using WebSockets
- Full payment integration for booking process
- Enhanced filtering and search capability for venues
- User analytics dashboard
- Upload/hosting optimization for venue images